CSE-355: Introduction to Blockchain Spring 2026

Programming Assignment 3 (Bonus)

Deadline: 11:55 pm on 4^{th} June 2026 **Total Marks:** 100

This assignment is based on the original work of Muneeb Ur Raheem and his team from LUMS. Full credit for the initial design and concept goes to them.

Preface: Building Your First Market on Blockchain

Welcome to **Programming Assignment 3**, where we take a major leap from writing simple smart contracts to building a **real-world economic mechanism**, the **Double Auction**.

In this assignment, you will not only code in Solidity, but also deploy and test your work on a locally hosted **Quorum blockchain network** using a pre-configured Virtual Machine. This assignment brings together all the essential skills you've built over the semester especially creating a smart contract in PA2 and interacting with a class Blockchain in PA1.

What You'll Experience:

- Design and implement a **Double Auction** mechanism as a Solidity smart contract.
- Deploy and interact with your contract using **Web3.js** through a JavaScript interface.
- Work on your **own blockchain node** configured within a dedicated Virtual Machine.
- Explore how **market logic, price discovery, and consensus** coexist in blockchain-based systems.

Learning Goals:

1. Understand how blockchain-based marketplaces are structured.
2. Gain hands-on experience in compiling, deploying, and interacting with smart contracts.
3. Learn how distributed nodes, consensus, and contract state work together.
4. Strengthen your debugging and testing workflow through automated scripts.

1 Introduction to the Double Auction algorithm

In this assignment we will be implementing a Double Auction algorithm. The basic idea is that you have a market – a forum where people wish to buy and sell **items**. The people trading in these items will submit their **bids** i.e. the price at which they are willing to buy or sell these items. These bids are submitted to a market institution or marketplace.

What is the job of this marketplace? This marketplace must **match** the sell bids and the buy bids. It collects bids in a time interval called the **Auction interval**. At the end of Auction Interval, the market picks a price called the market **clearing price**. This **clearing price** has some criteria but the most important is that no one trades their item at a loss. Let's assume that we have buy bids submitted by n buyers as well as sell bids submitted by m sellers. Here is the algorithm for how to match bids:

- Arrange the seller bids in ascending order: $s_1 ... s_m$. That is, s_1 is the smallest bid and s_m is the largest bid.
- Arrange the buyer bids in descending order: $b_1 ... b_n$, such that b_1 is the largest bid and b_n is the smallest bid.
- Find the **breakeven index** k. This k is the largest value such that $b_k \geq s_k$. We are finding the largest index where a buyer would be willing to buy from a seller.
- Calculate the clearing price as $\frac{b_k + s_k}{2}$. Match all the bids upto the breakeven index. Buy bids and sell bids upto the breakeven index get matched to each other i.e., $(b_1, s_1) ... (b_k, s_k)$. They make their trade at the clearing price. Buy bids and sells bids beyond the breakeven index do not get to trade. They aren't considered matches.
- The quantity of items, in case when the values are not the same for a matched pair of a buyer and a seller, is determined as the comparatively lesser number. For instance, if in a pair (b_i, s_i), the seller wanted to sell 8 units and buyer wanted to buy 10 units, 8 units will be traded via the Double Auction algorithm and vice versa.

Your job is to set-up this marketplace in a smart contract. You have already gained experience working with remix, so you understand how smart contracts work. In this assignment, you will go one step further and interact with smart contracts in JavaScript. Don't worry if this algorithm seems too abstract, we will cover it again after the setup guide. You can read more about it here if you want: `https://en.wikipedia.org/wiki/Double_auction#Average_mechanism`

2 A refresher on smart contracts

A smart contract can be defined as a computerized transaction protocol which automatically executes the terms of a contract when certain conditions are met. These conditions may be as simple as two accounts having a certain amount of balance but can get as complex as one would like. Smart contracts are visible to everyone on the blockchain and are thus everyone can verify whether the conditions were met and that the transactions were made. In this assignment, you will be working with smart contracts and you will be deploying them on a class blockchain that will be common between you and all your course mates. You will write some code in Solidity, compile it and deploy it to the blockchain. Let's look at a simple blockchain smart contract.

```
1  pragma solidity ^0.8.7; //use solidity version above 0.8.7 (upto but
       not 0.9.0)
2
3  contract SimpleStorage
4  {
5      uint private storedData; //a private variable LINE 5
6
7      constructor (uint initVal) public
8      { //have a constructor with the default variable initVal.
9        //it is public so other contracts (or our javascript code) can
       interact with it.
10       storedData = initVal;
11 }
12
13     function set (uint x) public
14     { //this is a public function, similar to the constructor.
15       //You can give it an input and it stores it in storedData
16       storedData = x;
17 }
18
19     function get() view public returns (uint retVal)
20     { //this function is of type 'view' so it doesn't cost gas.
21       return storedData;
22     }
23 }
```

Listing 1: SimpleStorage.sol

Pragma solidity 0.8.7 (LINE 1) specifies that the solidity compiler should have a version above 0.8.7 (upto 0.9.0). There is a contract called SimpleStorage which has a public unsigned integer variable storedData. There is a constructor which sets the initial value and a set and get function. There are some keywords there. The public keyword (LINE 7, LINE 13, LINE 18) specifies that the variable or function are accessible to other contracts in contrast with private variables/functions (LINE 5). The view type (LINE 18) specifies that the function does not cost gas and does not perform any changes to the blockchain. What is gas? Running any smart contract that changes the blockchain (i.e. stores values in variables or delete variables etc.) requires gas. Gas is an amount of ether that must be specified when anyone deploys the contract or interacts with the contract and anyone running the contract must pay this amount of ether. If you are performing more actions than your gas will allow, your actions will not be performed but you will still lose the gas amount you volunteered. You can use the set function to store a value in the variable storedData (LINE 7). You can use the get function (LINE 18) to get

the value. It is a view function as it doesn't make a change in the blockchain. You can actually use remix to look at the machine code for the contract shown above:

```
PUSH 80
PUSH 40
MSTORE
CALLVALUE
DUP1
ISZERO
PUSH [tag] 1
JUMPI
PUSH 0
PUSH 0
REVERT
...
```

Figure 1: Machine code for SimpleStorage.sol

As we can see, the solidity code in SimpleStorage.sol breaks into simple machine code like commands.

3 Virtual Machine Setup Guide

Instead of connecting to a remote server, you will be running the blockchain node locally on your own machine using a pre-configured Virtual Machine (VM). This ensures a consistent environment for compiling, deploying, and testing your smart contracts.

Prerequisites & Downloads:

- Download and install Oracle VirtualBox.
- Download and install 7-Zip (required for extracting the VM disk).
- Download the Assignment VM Disk (.7z)
- Watch the Video Tutorial for VM Setup

Step-by-Step VirtualBox Setup:

After downloading and extracting the VM disk zip file using 7-Zip, follow these steps to mount it in VirtualBox:

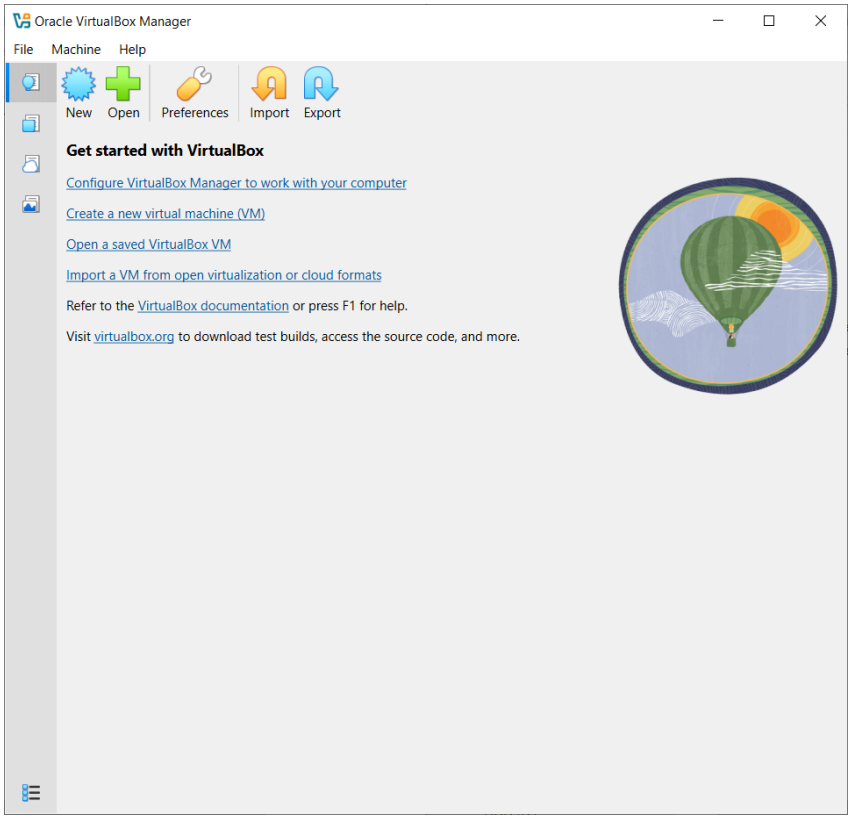

Figure 2: Oracle Virtual Box Manager initial screen. Click 'New'.

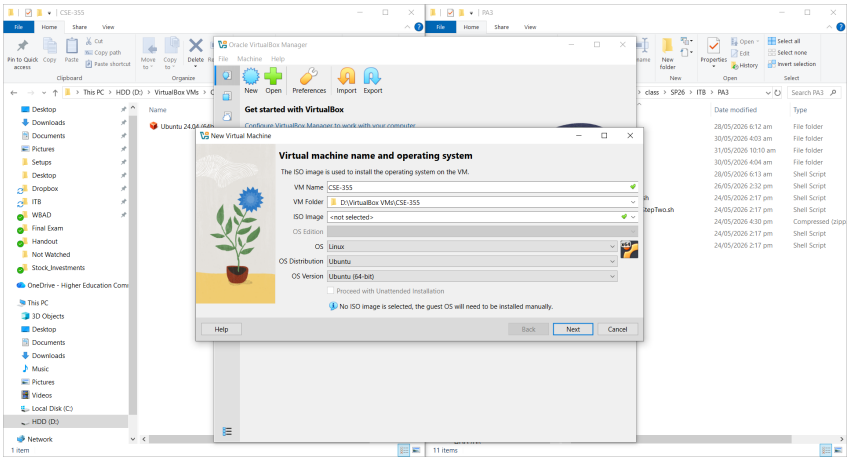

Figure 3: New Virtual Machine menu. Fill it out as shown.

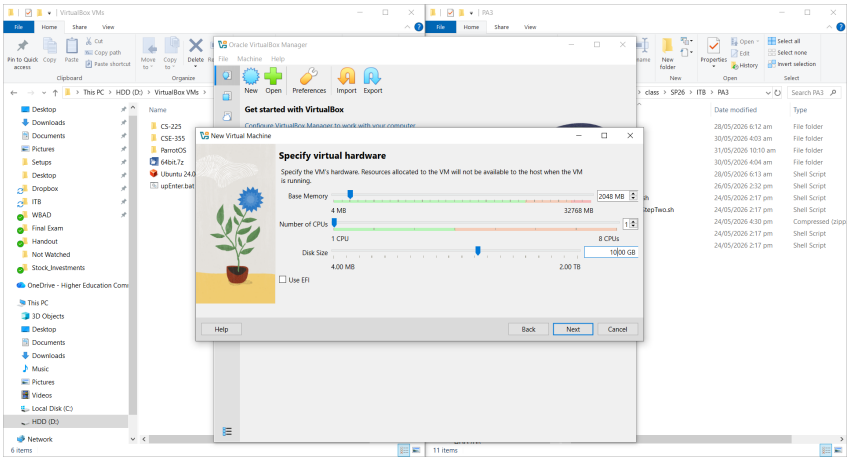

Figure 4: Specify Virtual Hardware. Select 2 GB RAM and 1 CPU. (The 10 GB disk size placeholder does not matter at this point).

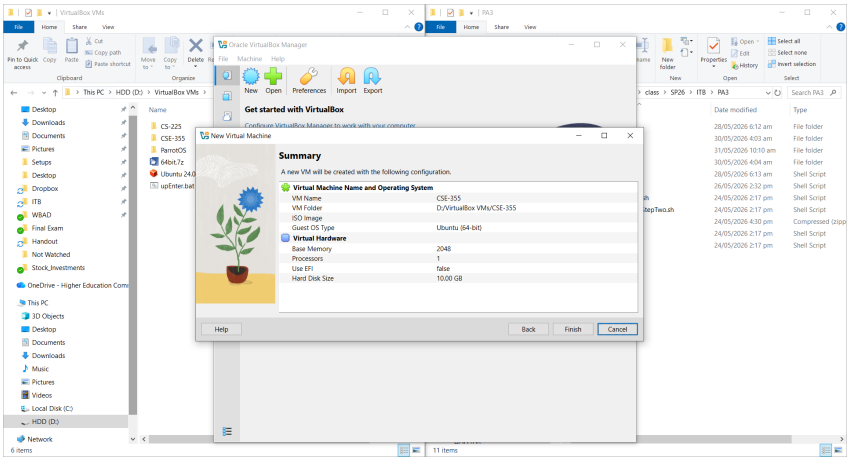

Figure 5: Summary screen. Click Finish to create the VM.

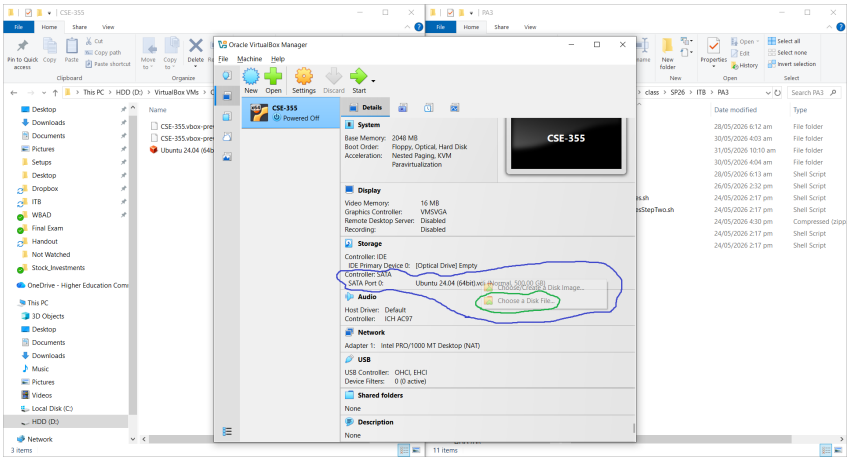

Figure 6: Go to the VM's Settings → Storage. Replace the default SATA Disk with the extracted VDI disk provided by the instructor.

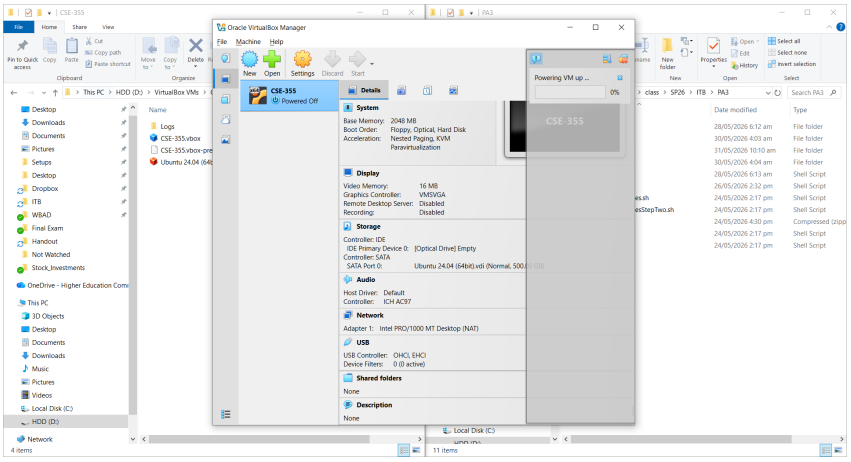

Figure 7: Click the 'Start' button to boot your newly configured VM.

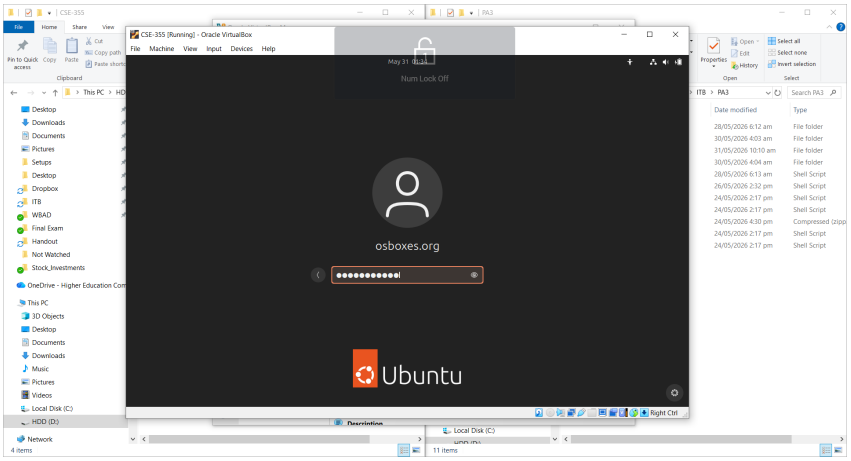

Figure 8: The Ubuntu login screen. The password is the same as the username: **`osboxes.org`**.

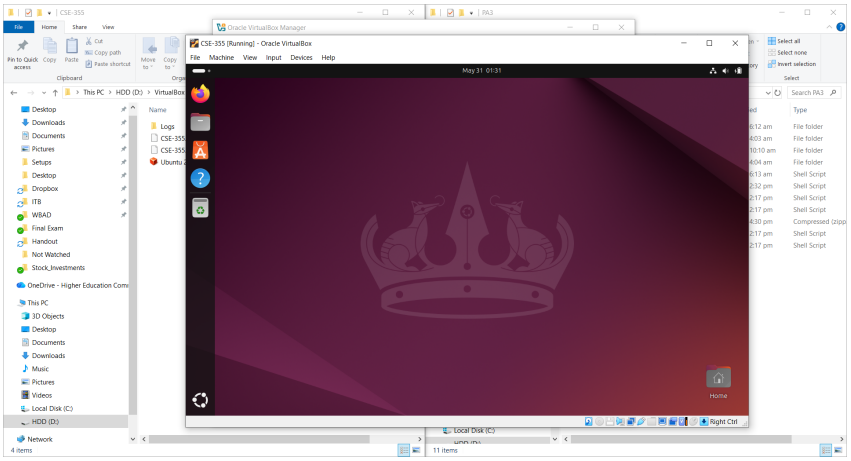

Figure 9: The Ubuntu desktop screen, ready for the assignment.

4 Manual Interaction with the blockchain

Now that your VM is running and you are on the Ubuntu desktop, it's time to start your blockchain node.

Starting the Node:

1. Open a terminal by pressing **`Left Ctrl + Alt + T`**. (Make sure to use the Left Ctrl key, as the Right Ctrl key is often reserved as the VirtualBox Host key).

2. Navigate to the default student directory:
   ```
   osboxes@osboxes:~$ cd ITB/PA3/students/pa3_std
   ```

3. Navigate to your node folder and execute the startup script:
   ```
   osboxes@osboxes:~/ITB/PA3/students/pa3_std$ cd Nodepa3_std
   osboxes@osboxes:~/ITB/PA3/students/pa3_std/Nodepa3_std$ ./startup.sh
   ```

The `startup.sh` script will automatically launch the required background validator nodes (which handle the consensus mechanism) and then start your specific student node in the foreground. Keep this terminal window open and running.

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/Nodepa3_std$ ./startup.sh
Starting consensus infrastructure (Validator Node0)...
Starting consensus infrastructure (Validator Node1)...
Starting consensus infrastructure (Validator Node2)...
Starting consensus infrastructure (Validator Node3)...
Starting consensus infrastructure (Validator Node4)...
Starting Student Node...
INFO [05-29|20:51:42.521] Running with private transaction manager disabled - quorum private transactions will not be supported
INFO [05-29|20:51:42.522] Maximum peer count                       ETH=50 LES=0 total=50
INFO [05-29|20:51:42.522] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
DEBUG[05-29|20:51:42.522] FS scan times                            list="63.585$\mu$s" set="9.335$\mu$s" diff="4.452$\mu$s"
TRACE[05-29|20:51:42.522] Started watching keystore folder         path=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/keystore
TRACE[05-29|20:51:42.523] Handled keystore changes                 time="187.984$\mu$s"
WARN [05-29|20:51:42.523] Using deprecated resource file /home/sysadmin/ITB/PA3/students/29008/Node29008/data/nodekey, please move this file
      to the 'geth' subdirectory of datadir.
WARN [05-29|20:51:42.523] Found deprecated node list file /home/sysadmin/ITB/PA3/students/29008/Node29008/data/static-nodes.json, please use
      the TOML config file instead.
DEBUG[05-29|20:51:42.523] Sanitizing Go's GC trigger               percent=100
WARN [05-29|20:51:42.523] WARNING: The flag --istanbul.blockperiod is deprecated and will be removed in the future, please use ibft.
      blockperiodseconds on genesis file
INFO [05-29|20:51:42.523] Enabling recording of key preimages since archive mode is used
INFO [05-29|20:51:42.523] Set global gas cap                       cap=25,000,000
INFO [05-29|20:51:42.523] Running with private transaction manager disabled - quorum private transactions will not be supported
INFO [05-29|20:51:42.523] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [05-29|20:51:42.523] Allocated cache and file handles         database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata cache=768.00MiB handles=524,288
DEBUG[05-29|20:51:42.535] Chain freezer table opened               database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata/ancient table=bodies items=0 size=0.00B
DEBUG[05-29|20:51:42.535] Chain freezer table opened               database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata/ancient table=receipts items=0 size=0.00B
DEBUG[05-29|20:51:42.535] Chain freezer table opened               database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata/ancient table=diffs    items=0 size=0.00B
DEBUG[05-29|20:51:42.535] Chain freezer table opened               database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata/ancient table=headers  items=0 size=0.00B
DEBUG[05-29|20:51:42.535] Chain freezer table opened               database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata/ancient table=hashes   items=0 size=0.00B
INFO [05-29|20:51:42.535] Opened ancient database                  database=/home/sysadmin/ITB/PA3/students/29008/Node29008/data/geth/
      chaindata/ancient readonly=false
DEBUG[05-29|20:51:42.536] Current full block not old enough        number=27808 hash=6d02db..8d1b0b delay=3,162,240
DEBUG[05-29|20:51:42.536] Account Extra Data root                  hash=000000..000000
...
```

Figure 10: The output of the startup.sh script, this should be updating regularly if your node is working. The contents don't matter too much.

The stuff isn't too relevant to us (yet) but what's important is that the node keeps displaying and updating this log.

Attaching to the Node:

1. Open a **second tab in the terminal window** (`Left Ctrl + Shift + T`).

2. Navigate back to the student folder:
   ```
   osboxes@osboxes:~$ cd ITB/PA3/students/pa3_std
   ```

3. We have created a few EOA's (Externally Owned Accounts) for you. You can view their passwords in the `web3data.json` file:

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std$ cat contracts/web3data.json
{ "location": "/home/osboxes/ITB/PA3/students/pa3_std/Nodepa3_std/data/geth.ipc", "
    password": "74b873" }
```

Figure 11: The contents of the web3data file that is present in the contracts folder

The password in Figure 11 is **74b873**. This is the password you will need to unlock any of your EOA and make transactions.

4. Attach to the running node using the JavaScript console:

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std$ geth attach Nodepa3_std/data/geth.ipc
Welcome to the Geth JavaScript console!
...
>
```

Figure 12: The JavaScript console

Once it is working, type **`net.peerCount`** to see how many peers you are connected to. You should also have some personal EOAs on this node, to check their addresses, type **`eth.accounts`**. Feel free to use tab on your keyboard to see (and play with) other commands you can execute.

```
> net.peerCount
5
> eth.accounts
["0x1da3afc4aff59707629676b7b97b6b3b466a8676", "0x4847c7d361b83586af33ce01999240dd962864d9"]
>
```

Figure 13: Some JavaScript console commands

Figure 12 is showing that this node has 5 peers (number of nodes this node is connected to); you may have many more peers (depending upon how many other students have started their node). Figure 13 is also showing the two EOAs (which are the only Ethereum accounts for this node). Your node may have more. You may also create new EOAs for yourself (using the ***personal.newAccount()*** command in the JavaScript console) but those EOAs will start off with zero ethers. Of course, you can transfer ethers to those new EOAs.

Notice that eth.accounts returns a list of your EOA addresses. The balance for an EOA can be checked either by using the index of that list or (less preferably) by copy/pasting the EOA address as: **`eth.getBalance("account address here")`**

We can transfer ethers from one address to another using: **`eth.sendTransaction({from: "senderaddress", to: "receiveraddress", value: 100000})`**

```
> eth.getBalance(eth.accounts[0])
10000000000000000000
> eth.getBalance(eth.accounts[1])
10000000000000000000
> eth.sendTransaction({from:eth.accounts[0], to:eth.accounts[1],value:1e5})
Error: authentication needed: password or unlock
                at web3.js:6355:37(47)
                at web3.js:5089:62(37)
                at <eval>:1:20(16)

>
```

Figure 14: Checking balance and attempting to send a transaction

Well, the transaction failed because the user wasn't authorized to act on the behalf of eth.accounts[0]. In order to be authorized, you must first unlock that account using a password as: `personal.unlockAccount("addressHere", "password here", time duration)`

The password used for unlocking is the same you may have obtained from the web3data.json file. Let's try it (Figure 15) and then send the transaction.

```
> personal.unlockAccount(eth.accounts[0],"d41d8c",120)
true
> eth.sendTransaction({from:eth.accounts[0], to:eth.accounts[1],value:1e5})
"0x559faae23913d174ec7d2f170d1cead600339be1682853f6b8c48980ffe8d5e3"
> eth.getBalance(eth.accounts[0])
9999999999999900000
> eth.getBalance(eth.accounts[1])
10000000000000100000
>
```

Figure 15: Unlocking an account and successfully sending a transaction

It works and you have sent the ether in a successful transaction. It may take several seconds for the transaction to be validated.

Note: do not send your transactions to any address you don't trust. This may not be real currency but you will still need it to run your smart contracts. We can't give you more even if we wanted to so experiment with small amounts; your starting balance is 10 ethers which should be enough. If you still end up broke, ask us and we may become generous.

Press **ctrl+d** to exit the JavaScript console. (It's probably best to actually try everything till here before moving on, otherwise it might get overwhelming).

5 Updating Node Modules Important

Before Deploying your smart contract and running test cases you need to update your Node Modules found in `package.json` inside the `contracts` directory.

Follow these steps to update your Node Modules (you only have to do it just once):

1. First of all, cd to contracts: **`cd contracts`**
2. Enter the command to update Web3 packages: **`npm install web3@1.6.0`**
3. Then enter the command to update Solidity packages: **`npm install solc@0.8.9`**

This is because these versions are necessary to be installed in the provided VM for things to work, otherwise the deployment may fail.

6 Programmatic Interaction with the blockchain

Let's navigate to the contracts folder.

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std$ cd contracts/
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ ls
compile.js deploy.js interact.js location.json node_modules package.json package-lock.json SimpleStorage.sol
     web3data.json
```

Figure 16: The contents of the contracts folder

Below are some of the files you will **NOT** be using (but feel free to look at them)

- web3data.json: this file contains the location of your ipc file and the passwords for your EOAs (that we made for your convenience). Do **NOT** modify or delete this otherwise you will probably be unable to attempt this assignment.
- node_modules, package.json, package-lock.json are all used by JavaScript.

Do not delete or alter any of the files above. If you delete these files, your particular node may stop working. We really really can't help you then as the files were generated when we started the blockchain. Please be careful.

Now, the important files:

- **SimpleStorage.sol** has your contract
- **compile.js** uses a solidity compiler to compile "SimpleStorage.sol" and make a file called **SimpleStorage.json** that has the abi and bytecode for the smart contract. Important note: It's best to actually test your code on remix beforehand; this step will give you a nasty error if your code doesn't compile for any reason (for example, syntax errors).
- **deploy.js** deploys the json file to the blockchain. It uses the ipc file and web3 to do so. Then, it puts the address of the deployed contract in a file called "contAddress.json".

- **interact.js** This is probably the most important file. The code in this file allows you to **programmatically** interact (in contrast with the manual interaction done through the javascript console) with the deployed contract's functions such as the set or get function in the deployed contract (whose code was given in Listing 1, earlier in the handout). Try **`nano interact.js`** to see/edit the contents of this file and read the comments.

In your directory, the JavaScript files are ready to work with the contract in Listing 1. Let's try executing these (using the command **`node whatever.js`**) and see what happens.

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node compile.js
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node deploy.js
IPC file is located at: /home/osboxes/ITB/PA3/students/pa3_std/Nodepa3_std/data/geth.ipc
Your account is:  0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2
Account unlocked!
The transaction hash is: 0x6ae8d6e32d245bdfaafef00928dc008e5336bd90afae34cae336df238381bb07
contract address 0x3Af0587336B6101526E96000f14004EDe84BB8F8
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node interact.js
IPC file is located at: /home/osboxes/ITB/PA3/students/pa3_std/Nodepa3_std/data/geth.ipc
the contract is at:  0x3Af0587336B6101526E96000f14004EDe84BB8F8
Your account is:  0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2
Account unlocked!
value at the contract currently is:  47
value at the contract currently is:  100
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$
```

Figure 17: Deploying and interacting with the blockchain

Here's is what's happening. The compile.js is just compiling the contract and putting it in the same directory. The deploy.js is deploying the contract and telling us what the contract address is. The interact.js is changing the value in the storedData variable from 47 to 100. It reads the value before and after calling the set method in the contract.

You are highly encouraged to look at compile.js, deploy.js and interact.js

Let's go through a simple example of changing the code. The changed code is:

```
1  pragma solidity ^0.8.7;
2
3  contract SimpleStorage
4  {
5      uint private storedData;
6      address senderAddress; // another variable, "address" is a datatype
       in solidity
7
8      constructor (uint initVal) public
9      {
10         storedData = initVal;
11     }
12
13     function set (uint x) public
14     {
15         storedData = x;
16         senderAddress = msg.sender; // put the value of the account who
       called this function
17     }
18
19     function get() view public returns (uint retVal, address retAdd)
20     {
21         return (storedData, senderAddress);
22     }
23 }
```

Listing 2: Modified SimpleStorage.sol

It has been altered (LINES 6, 16, 19 and 21). There is another address variable called "senderAddress" (declared in LINE 6). The set function now keeps track of the person who sent the message by calling the function (LINE 16). The get function is also changed and now returns the latest person who called set and along with the original `uint` value as a tuple (LINE 21). Notice how the return type is changed in the function header as well (LINE 19).

As the return type from the get function is changed, we need to change the relevant bits of code in `interact.js` as well. It was originally

```
  // call the setter function
  reading = await get()
  console.log('value at the contract currently is: ', reading)

  await set(myAccount, Math.floor(100 + Math.random() * 1000)) //set
   the value to a random number between 100 and 1100

  reading = await get()
  console.log('value at the contract currently is: ', reading)
```

Listing 3: `interact.js` snippet

We have to change it to something like

```
// call the setter function
reading = await get()
console.log('value at the contract currently is: ', reading.retVal,
   reading.retAdd)

await set(myAccount, Math.floor(100 + Math.random() * 1000)) //set the
   value to a random number between 100 and 1100

reading = await get()
console.log('value at the contract currently is: ', reading.retVal,
   reading.retAdd)
```

Listing 4: modified `interact.js` snippet

Notice how when there was only one value being returned by the get function, we got a single variable but in our newer contract, we are receiving two values. In that situation, you are getting a structure that has both those values as separate fields and the name of those fields is the same as the one we defined in our contract (retVal and retAdd). The names of the fields are available from the abi file (something similar to a header file) that you had generated when executing compile.js. It is instructive to go through the abi file (currently configured to be included within the SimpleStorage.json file but may also be generated as a separate file).

Let's compile, deploy and interact with this.

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node compile.js
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node deploy.js
IPC file is located at: /home/osboxes/ITB/PA3/students/pa3_std/Nodepa3_std/data/geth.ipc
Your account is:  0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2
Account unlocked!
The transaction hash is: 0xd4148ca3ea17080e7dacb036c49e77ad009bf9af47c3ebc5549229d1b0133777
contract address 0xe41CbE2F4761b0D7fA98fDdCd6bD6391271D7C4A
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node interact.js
IPC file is located at: /home/osboxes/ITB/PA3/students/pa3_std/Nodepa3_std/data/geth.ipc
the contract is at:  0xe41CbE2F4761b0D7fA98fDdCd6bD6391271D7C4A
Your account is:  0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2
Account unlocked!
value at the contract currently is:  47 0x0000000000000000000000000000000000000000
value at the contract currently is:  643 0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$
```

Figure 18: Interacting with the modified smart contract. It now prints an integer and an address

As you can see, the contract has an initial value of 47 and "0x00.." because set hasn't been called before but afterwards, it is 643 and "0x1D..." which is the address of the first account (EOA) you have. Try writing and uploading your own contracts and interacting with different EOAs that you have.

7 Your task: a double auction contract

As described in the start, you will be implementing a Double Auction algorithm. In specific, you will be implementing an average mechanism [1]. You will be writing a smart contract that acts like a marketplace. The smart contract will have functions to add buy bids or sell bids as well the option to call the Double Auction and create matches. The scenario:

1. You have multiple buyers and sellers who are submitting bids.
2. Each bid is a pair of (quantity, value).
3. Each account (EOA) address is a potential buyer/seller. You have 12 accounts (EOAs).
4. Whenever the double auction happens, the buyers and sellers are matched. You will need to ensure that the double auction happens no more than once in a 30-second period (called the auction interval).
5. An address/EOA can't submit multiple bids in the same auction interval.
6. A user should be able to access the results of the auction held at the last occurrence of the double auction.

The double auction algorithm works as follows:

1. Arrange the sellers in ascending order of value. Arrange the buyers in descending order of value.
2. compare the sorted lists of buyers and sellers. Assume the sorted values for buy bids are $b_1 \dots b_n$ (descending) and for the sell bids are $s_1 \dots s_m$ (ascending). Find the **maximum** index k such that $b_k \geq s_k$. This number k is called the breakeven index. If no breakeven index is found, then the Auction is not called and all the bids are rejected. This may happen if all sell bids are higher than buy bids or if the number of buy/sell bids are zero.
3. For all buyers/sellers with index $> k$, ignore the bids. They are a bad match. For all buyers/sellers with index $\leq k$, carry out the transaction at a price $= \frac{s_k + b_k}{2}$. Keep the quantity as the **minimum** of the buyer's and seller's quantity.

[1] `https://en.wikipedia.org/wiki/Double_auction#Average_mechanism`

Your smart contract should have 4 main required public functions (and as many others as you want as helpers):

- **addBuyer**: This should take in a bid and add it to the list of buy bids the contract is tracking. Make sure the address of the account (EOA) submitting the bid is not already present in a buy or sell bid that has been placed in the same auction interval. The contract must check if the buy bidder has enough balance to buy the required quantity; else, you can revert with an error statement of your choice. (5 marks)

- **addSeller**: This should take in a bid and place it in the list of sell bids the contract is tracking. Make sure the address of the submitting EOA is not already present in a buy or sell bid in the same auction interval. There is no need to check if the seller has the quantity they specified in their bid. This is because we are not tracking the item quantity. (5 marks)

- **doubleAuction**: Any user (including those who placed a buy or sell bids) can invoke this function. This performs the double auction and creates some lists of addresses (buyers $\rightarrow$ sellers) with respective quantities and auction price. Note that this function should only be callable once every 30 seconds in real time. If it is called more often, then it should return without doing anything, perhaps reverting with an error message. (40 marks)

- **getResults**: This function returns the lists of buyer addresses, seller addresses, quantities, and auction price. You can also return additional things if you want. (10 marks)

  Please make sure that the order of the bids is intact. Index 1 corresponds to the seller with lowest value and the buyer with the highest offer.

In addition to these, you must write the corresponding JavaScript code for each of the above functions. The javascript files (discussed next) are already present, just fill in the code wherever appropriate. These javascript files will allow you to programmatically interact with your (or someone else's) smart contract.

- **addBuyer.js (5)**: The script takes in value, quantity and account number as a command line input. It should output `submitted a buy bid of: ( q , v ) from account: 0x12345` where q is the quantity, v is the value and 0x12345 is the address.

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node addBuyers.js 2 3 0
submitted a buy bid of: (2 , 3) from account: 0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2
```

Figure 19: Submitting a buy bid of quantity 2, value 3 from account number 0

- **addSeller.js (5)**: The script takes in value, quantity and account number as a command line input. It should output `submitted a sell bid of: ( q , v ) from account: 0x12345`.

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node addSeller.js 4 5 1
submitted a sell bid of: (4 , 5) from account: 0x850c9d2460f95f0D8f8DD1ab2E6F84d5e21B97D3
```

Figure 20: Submitting a sell bid of quantity 4, value 5 from account number 1

- **doubleAuction.js (15)**: Calling this script will, in turn, call the double auction function in the contract. It outputs "Double Auction Successful" if the double auction function in the contract was successfully called (when, for example, the auction interval had passed).

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node DoubleAuction.js
Double Auction Successful
```

Figure 21: Calling Double Auction Successfully

Otherwise, this script outputs "Double Auction not Successful".

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node DoubleAuction.js
Double Auction not Successful
```

Figure 22: Calling Double Auction Unsuccessfully

- **getResults.js (15)**: This script should print the results (if any) of the latest successful double auction. If there was a successful double auction with at least one match, it should print the following: `index sellAddresses buyAddresses C Q`

  If there has been no successful auction OR there were no matching bids in the latest successful auction, it should **print nothing at all**. Where C is the clearing price and Q is the quantity transferred. The entries should be separated by a space or a tab. Each new entry should begin on a new line. Make sure that the bids are in order i.e., index 1 corresponds to the seller with lowest sell bid value and the buyer with the highest buy bid value. **Formatting is important!**

```
osboxes@osboxes:~/ITB/PA3/students/pa3_std/contracts$ node getResults.js
index    sellAddresses                              buyAddresses                                C       Q
1        0xd13008857C3Cafddd18b20045cE9B73BF7E99D69      0x56Dc4e50285E3a76718b6d6F6292F183a77022dc      6       1
2        0x850c9d2460f95f0D8f8DD1ab2E6F84d5e21B97D3      0x84CF327cd406e6Df4e3F17b079084461067e6a7a      6       2
3        0x6c4a358Db9c674806664aD5B569d399ccEA0F68a      0xe8A1E3C2b703Ed8AC5AE095A9113f896E97e3353      6       3
4        0xc20f9Aa1c0208d5EEdC597CfaB13252c924444e2      0x8E8e89F0F92782ff9859A737Ddc811174e22A21E      6       5
```

Figure 23: Output of **`getResult.js`** After Successful Double Auction

We have already written the script in **`compileDoubleAuction.js`** and **`deployDoubleAuction.js`**. You will probably not have to change that but don't let that stop you.

8 Testing and Potential Problems

We have provided you with the test cases located in **`testResult.sh`**. These same test cases will be used for grading your assignment. To execute the test cases, run the command: **`bash testResult.sh`**

Please ensure that you are running startup.sh **on seperate terminal while running the test cases.**

The test cases will automatically compile and redeploy your smart contract onto the chain. They will call your JavaScript files to submit bids and then execute the Double Auction algorithm. We will assess the outputs of your Double Auction algorithms and bids by calling **`getResult.js`** and comparing its output. We advise you to manually deploy your smart contract onto the chain before running the test cases.

Some potential problems you may face are:

1. Have the name of the contract and the .sol file be exactly the same otherwise compile.js (and compileDoubleAuction.js) will fail.

2. Read interact.js to find out how .call and .send are different. Don't try to return a value to JavaScript from a solidity function that also takes in an input.

3. Keep an eye on the log of the node. If it stops and/or returns to command line, run **`bash startup.sh`** in the appropriate folder again.

4. Don't waste all your ether accidentally. If you send it to an address with a typo, it will be gone forever.

5. If you are using remix to test your solidity code (you really should try), then make note that calling functions that have block.timestamp will result in an error.

6. Don't delete/modify the files in the Nodepa3_std folder, you might end up getting disconnected from the rest of the blockchain permanently.

7. The formatting in the output results from **`getResults.js`** is very important. **`testResults.sh`** in your **`contracts`** folder has a simple string comparison so try to stick to the instructions.

8. If testResult.sh is taking significant time to execute and seems to hang then try making sure first that your smart contract is properly deployed over the chain.

   (a) First of all, **`cd`** to **`contracts`**.

   (b) Enter the command to compile your smart contract: **`node compileDoubleAuction.js`**

   (c) Then enter the command to deploy your smart contract: **`node deployDoubleAuction.js`** If you see error such as "The Contract couldn't be stored. Please check your gas limit". Then make sure to update your node modules as explained in section 5.

9. Ensure that any Javascript code that requires gas to interact with your Smart contract is provided the same gas limit as in deploying your smart contract, which would be **`0xe00000`**.

In addition, the following links might be useful

- `https://docs.soliditylang.org/en/v0.8.13/`
- `https://web3js.readthedocs.io/en/v1.3.4/web3.html`
- `https://en.wikipedia.org/wiki/Double_auction#Average_mechanism`

8.0.1 Debugging Tips

- Before starting your assignment, it is advised that you save a copy of contracts and Node folder locally so that if you accidentally corrupt a file, you have a backup.
- Before deploying it on the class blockchain, you should first code and deploy the double auction contract on solidity for sanity check.
- If your test cases keep failing despite correct working of the contract through manual checking (on remix), there is probably a formatting problem. In that case, you can modify the **`testResults.sh`** file to show you what your function call is returning and what the test file is actually accepting.

8.1 Submission instruction

You must upload the following files to LMS in the appropriate tab and submit the assignment:

- DoubleAuction.sol
- addBuyers.js
- addSeller.js
- DoubleAuction.js
- getResults.js